CNN 합성곱 신경망 (Convolutional Neural Network)

작은 필터 하나를 그림 전체에 재사용한다

Machine Learning and Deep Learning

KSA 2027-Spring
Sumin Han (suminhan@ksa.hs.kr)

오늘의 질문

손글씨 숫자 하나 (28 × 28 픽셀) 를 알아보는 데
가중치 **125만 개**가 꼭 필요할까?

완전연결: 픽셀 784개를 뉴런 1,600개에 전부 연결 $784 \times 1{,}600 + 1{,}600 = 1{,}256{,}000$

완전연결의 두 가지 문제

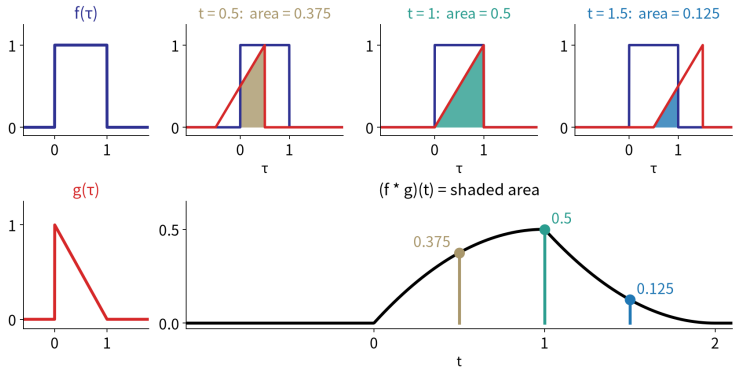
가중치가 너무 많다 그림이 커지면 폭발 (224 × 224 컬러면 입력만 15만 개)

위치를 모른다 같은 가로획이 한 칸 옮겨지면 **완전히 다른 입력**으로 본다

이웃한 픽셀끼리 가깝다는 사실을 쓰지 않는다

그림의 성질: **가까운 픽셀끼리 의미가 있다** **같은 모양은 어디에 있든 같은 모양**

합성곱 $f * g$: 하나를 뒤집어 밀면서 겹친 넓이를 잰다



$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)\, g(t-\tau)\, d\tau$$

이산: $(f * g)[n] = \sum_{k} f[k]\, g[n-k]$

$g(t-\tau)$ = g 를 **뒤집어** t 만큼 민 것 겹친 곳에서 곱한 넓이 = 그 위치의 출력

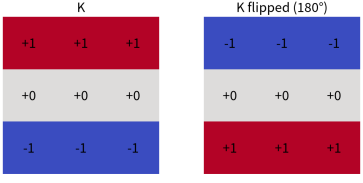
그림에서는 2차원: output $= I * K$

수학의 합성곱 (필터를 뒤집는다)

$$(I * K)[i,j] = \sum_m \sum_n I[i-m,\ j-n]\ K[m,n]$$

CNN 이 실제로 하는 계산 (**뒤집지 않는다**, 교차상관 $\star$)

$$(I \star K)[i,j] = \sum_{m=0}^{2} \sum_{n=0}^{2} I[i+m,\ j+n]\ K[m,n]$$



I = image (입력 그림), K = kernel (필터)

뒤집으면 가로선 필터는 **부호만** 바뀐다

필터 값은 **학습**으로 정하니 뒤집든 말든 같은 일

$\Rightarrow$ 딥러닝은 $\star$ 를 그냥 **합성곱**이라 부른다

실제 사진에 필터를 바꿔 끼우면

original



$*\ 1/9$

1	1	1
1	1	1
1	1	1

$=$

blur (mean)



original



$*$

0	0	0
0	0	1
0	0	0

$=$

shifted left by 1 pixel



original



$*$

0	-1	0
-1	5	-1
0	-1	0

$=$

sharpen



original



$*$

1	1	1
0	0	0
-1	-1	-1

$=$

horizontal edges (|value|)

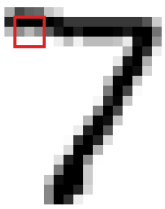


그림은 같고 **필터 숫자 9개만** 바꿨다 흐리게 · 옮기기 · 선명하게 · 모서리

여기서 $*$ = CNN 의 합성곱 (뒤집지 않음 (p.4)) 테두리는 0 으로 채워 크기 유지 (80×80)

합성곱: 3 × 3 필터를 겹쳐 곱하고 더한다

손글씨 7 (28 × 28)



빨간 창의 픽셀

1.0	1.0	1.0
0.4	0.3	0.4
0.0	0.0	0.0

가로선 필터 (3 × 3)

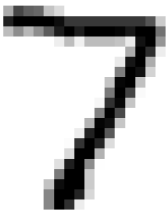
+1	+1	+1
+0	+0	+0
-1	-1	-1

$$(1+1+1)\times(+1)+(0.4+0.3+0.4)\times 0+(0+0+0)\times(-1)=\mathbf{3.0}$$

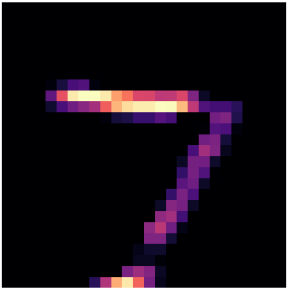
위가 밝고 아래가 어두우면 큰 값 = 가로선의 윗변 필터 = 숫자 9개

필터를 한 칸씩 옮기며 그림 전체를 훑는다

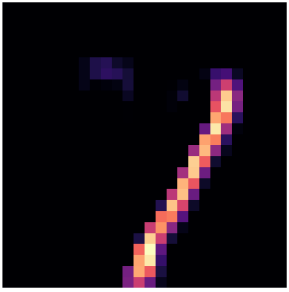
입력 28 × 28



가로선 필터 → 26 × 26



세로선 필터 → 26 × 26



특징 지도 (feature map): 가로선 필터는 가로획이, 세로선 필터는 **내려오는 획**이 밝다

어느 위치에서든 **같은 9개**를 쓴다 = 파라미터 공유 같은 모양은 어디에 있든 같은 반응

출력 크기

$$\text{출력} = \left\lfloor \frac{n - k + 2p}{s} \right\rfloor + 1$$

$n = 28$, $k = 3$, 패딩 $p = 0$, 보폭 $s = 1$	가장자리 한 칸씩 줄어든다	**26**
$p = 1$ (가장자리에 0 을 한 겹)	크기 유지 (“same”)	28
$p = 1$, $s = 2$ (두 칸씩 건너뛰기)	절반으로	14

k: 필터 크기 p: 패딩 (padding) s: 보폭 (stride)

채널과 가중치 수

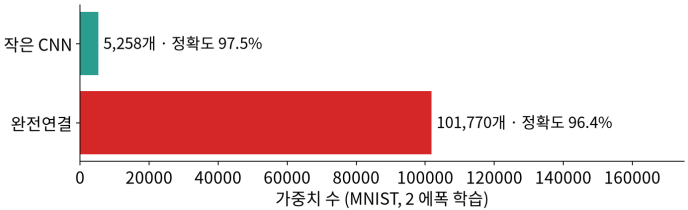
입력 채널 c_{in} 개 (컬러면 3) $\rightarrow$ 필터 하나 = $k \times k \times c_{\text{in}}$ 필터 c_{out} 개 = 출력 채널

$$\text{가중치} = (k^2\, c_{\text{in}} + 1)\, c_{\text{out}}$$

1층: $1 \rightarrow 4$ 채널	$(9 \cdot 1 + 1) \times 4$	40
2층: $4 \rightarrow 8$	$(9 \cdot 4 + 1) \times 8$	296
3층: $8 \rightarrow 16$	$(9 \cdot 8 + 1) \times 16$	1,168
합계		**1,504**

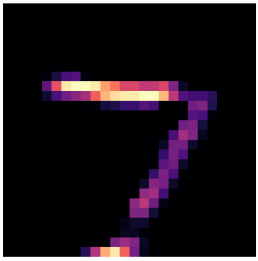
1,504 개 vs 1,256,000 개

CNN 3층 **1,504개** vs 완전연결 **1,256,000개** (835배)



실제 학습: 작은 CNN **5,258개로 97.5%** 완전연결 101,770개로 96.4%

풀링: 작게 줄이면서 특징은 남긴다





최대 풀링: 2 × 2 중 **가장 큰 값**만 26 × 26 → 13 × 13 가중치 0개

크기는 $\frac{1}{4}$, 계산량도 줄어든다 획이 한 칸쯤 움직여도 결과가 비슷하다

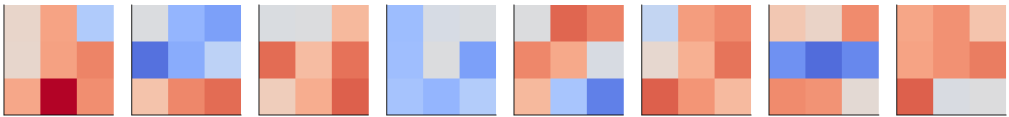
전형적인 CNN 구조

$$\text{입력} \to [\text{합성곱} \to \text{ReLU} \to \text{풀링}] \times \text{몇 번} \to \text{펼치기} \to \text{완전연결} \to \text{출력}$$

앞쪽 층	선 · 모서리 같은 작은 특징
뒤쪽 층	앞 특징의 조합: 고리, 획의 교차, 눈 · 바퀴 같은 부분
마지막	완전연결로 “이 특징들이면 7”

LeNet-5 (LeCun 외, 1998): 우편번호 숫자 인식 AlexNet (2012): 이미지넷 1위, 깊은 CNN 시대의 시작

학습된 첫 층 필터



MNIST 로 학습한 작은 CNN 의 첫 층 필터 8개 (빨강 +, 파랑 −)

사람이 정해 주지 않았는데 **가로 · 세로 · 대각선 모서리**를 찾는 모양이 된다

문제풀이

1. 32×32 그림에 5×5 필터, 패딩 0, 보폭 1 이면 출력 크기는? 패딩 2 면?
2. 입력 3채널 (컬러), 3×3 필터 64개인 합성곱 층의 가중치 수는?
3. 픽셀 값 $\begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$ 에 가로선 필터 (위 +1, 가운데 0, 아래 -1) 를 쓰면?
4. 특징 지도 $\begin{bmatrix} 1 & 3 \\ 2 & 0 \end{bmatrix}$ 에 2×2 최대 풀링을 하면?

실습

1. 합성곱 · 최대 풀링 직접 구현 → 손글씨 7 에서 3.0, 26×26, 13×13 재현
2. 출력 크기 · 가중치 수 계산 함수 → PyTorch 와 비교
3. 작은 CNN vs 완전연결 (MNIST): 가중치 수와 정확도, 학습된 필터 그리기

MNIST (LeCun, Cortes, Burges; torchvision)

▶ Colab에서 실습 노트북 열기

정리

합성곱: 작은 필터를 겹쳐 곱하고 더하기, 그림 전체를 훑어 특징 지도

파라미터 공유: 어디서든 같은 필터 $\rightarrow$ 1,504 vs 1,256,000 (835배)

출력 크기 $\frac{n-k+2p}{s}+1$ 가중치 $(k^2 c_{\text{in}}+1)c_{\text{out}}$

최대 풀링: 2×2 중 최댓값, 크기 $\frac{1}{4}$ 필터는 학습으로 모서리를 찾는다

이미지 출처

- 사진 (p.5): Cameraman, scikit-image `skimage.data.camera()` (Library of Congress, no known copyright restrictions)
- 손글씨: MNIST (LeCun, Cortes, Burges; torchvision)